What is Claude Code?

What is Claude Code?

Claude Code is an agentic coding tool that understands your codebase, edits your files, runs commands, and integrates with your existing developer tools to help you get things done faster. It's available in your terminal, Visual Studio Code, the Claude Desktop app, on the web, and in JetBrains IDEs.

What Separates Claude Code from Claude?

If you've used Claude.ai before, you might be wondering what makes Claude Code different. Unlike Claude.ai, Claude Code has direct access to your files, your terminal, and your entire codebase. Instead of copying and pasting code back and forth, it goes in and does the work itself.

The key differentiator is that Claude Code works as an AI Agent.

What is an Agent?

An AI Agent is software that can interact with its environment and perform actions to complete a defined goal. At its core, this works by having a large language model operating in a loop in real time. AI Agents can have access to tools, external services, or even other AI Agents to help reach their goals.

What Can Claude Code Actually Do?

Here's what that looks like in practice:

- Read and understand your codebase. You can ask Claude Code to explain a feature or trace a bug throughout your code.
- Edit files across your project. Claude Code can refactor a function and update every file that references it.
- Run terminal commands. It can execute your build script, run your tests, install packages, and use the output to decide what to do next.
- Search the web. If it needs documentation or the latest API references, it can look that up for you.

Using Claude Code Effectively

To use Claude Code effectively, keep these three concepts in mind:

The context window. Think of this as Claude's working memory. It can hold a lot, but not everything at once. This is where the "agentic" aspect comes in — Claude finds strategic ways to locate answers within your codebase without loading the entire thing into context.

It asks for permission. By default, Claude Code will ask you before running commands or making changes. You're always in control, whether you prefer a hands-on or hands-off approach.

It can make mistakes. Just like any tool, Claude Code isn't perfect. It might misunderstand your intent, introduce a bug, or over-engineer a solution. Staying in the loop helps you catch these early.

Recap

Claude Code is an agentic coding tool. It reads your codebase, edits your files, runs commands, and connects to external tools to help you ship faster. You can use it today in your terminal, VS Code, JetBrains, and the Claude Desktop app.

Video

How Claude Code Works

How Claude Code Works

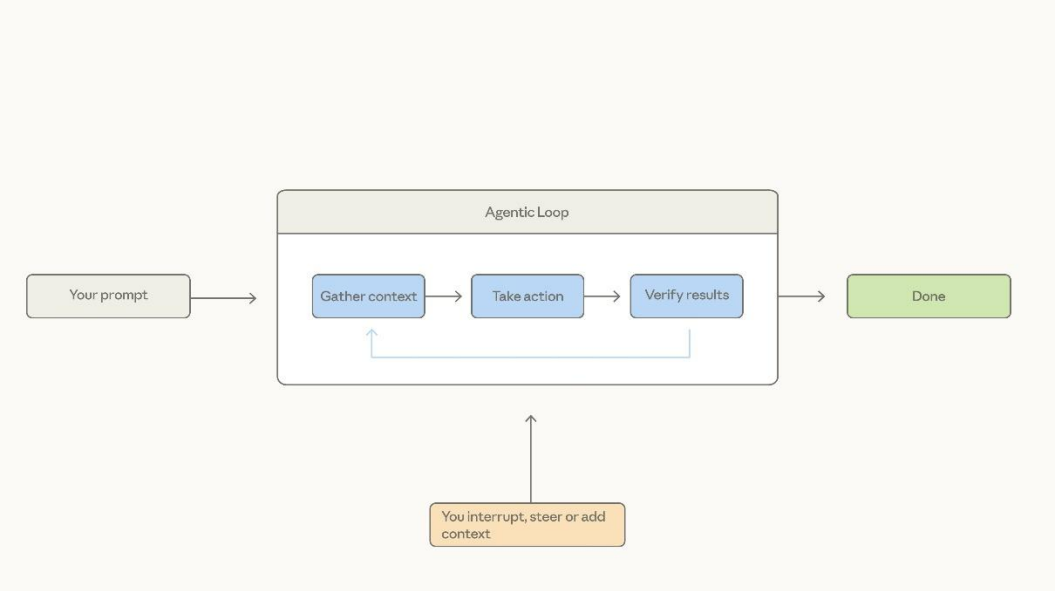
Claude Code is different from typical chat applications. Understanding how it works under the hood will help you use it more effectively.

The Agentic Loop

Claude Code is best explained through the agentic loop:

1. You enter a prompt into Claude Code.
2. Claude gathers the context it needs by interacting with the model, which returns text or a tool call that Claude Code can execute.
3. It takes action — for example, editing a file or running a command.
4. It verifies the results and determines whether they achieve what your prompt set out to do.
5. If they do, Claude finishes and waits for the next prompt. If they don't, it loops back and tries again until the results are complete and verifiable.

Throughout this loop, you can add context, interrupt, or steer the model to help guide it toward your goal.



Context

Claude has a context window that determines how much of your conversation, file contents, command outputs, and more it can store and reference. Once you reach that limit, Claude Code compacts your conversation — automatically determining what it can remove or summarize to bring the context window back down to a usable size.

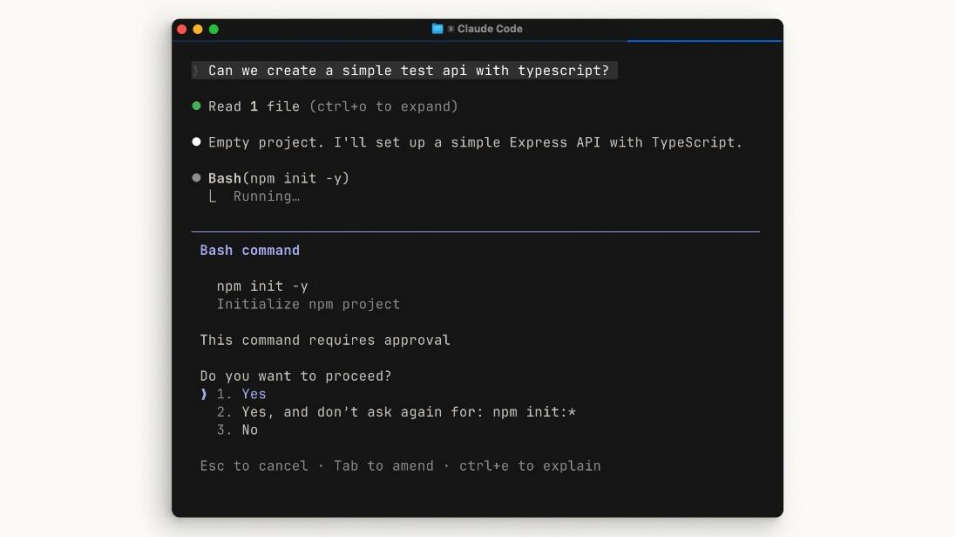
Tools

Tools are the backbone of how agents work. Most AI assistants simply take text in and return text out. Tools let Claude Code determine *when* to execute code to get closer to completing a task. This could be a file-reading tool, a web search tool, or any number of other capabilities. Claude Code uses semantic understanding to determine when to call a tool and how to use the output.

Permissions

Claude Code has several permission modes:

- Default behavior: Claude asks for explicit permission before editing a file or running a shell command.
- Auto-accept: Files are edited without asking, but commands still require approval.
- Plan mode: Uses read-only tools to compile a plan of action before starting any work.



All of this can be configured in your settings file. Be cautious when skipping permissions — giving Claude Code free rein to run commands means a mistake could be harder to catch before it happens.

Recap

Claude Code combines several agentic concepts: an agentic loop, a managed context window, tools, and configurable permissions — all inside your terminal. It can read your codebase, take action, and verify its own work. That's what makes it fundamentally different from a chat window.

Your first prompt

Installing Claude Code

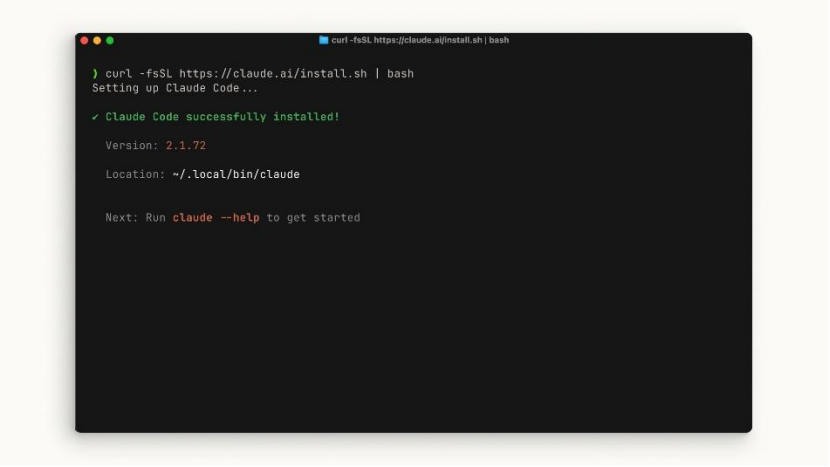
[Installing Claude Code]

Claude Code is simple to install whether you want to use it in your terminal, on the web, or in your IDE.

Terminal

On macOS, Linux, or WSL, use the curl command to install it in one go. If you prefer Homebrew, you can also use brew install, but note that this method doesn't support auto-updates.

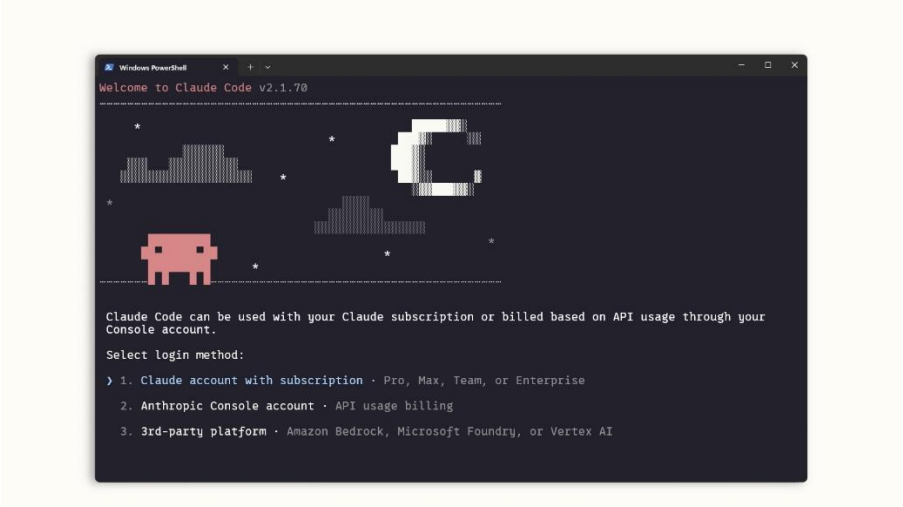
On Windows, there are a few options. In PowerShell, use the Invoke-RestMethod command. In CMD, use the curl command. There's also a winget command available, though like Homebrew, it won't auto-update.



After installation, you should be able to run the claude command. If not, restart your terminal. Navigate to your project directory and run:

claude

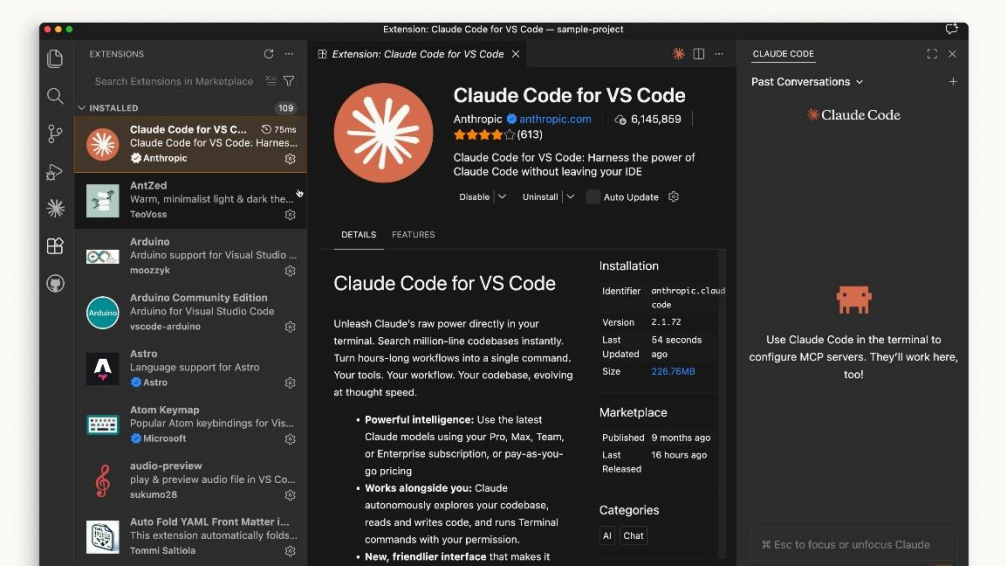
You'll go through some initial setup steps like choosing your color theme and signing in with your Claude account (Pro, Max, or Enterprise) or using an API key. If your organization has a Claude Enterprise account, be sure to select that option.



Whatever directory you run claude in, it will have access to that directory and all of its subfolders.

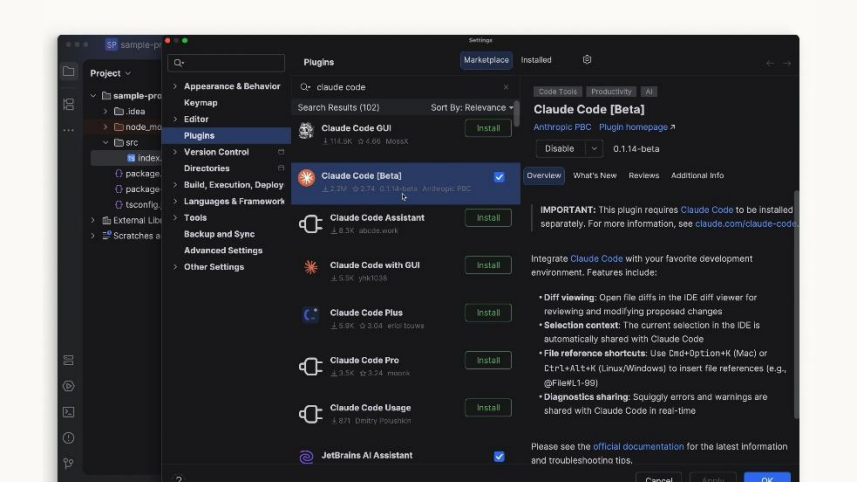
Visual Studio Code: Open your Extensions panel and search for "Claude Code." Look for the extension by Anthropic with the blue verification check. Hit install.

After installation, you may need to restart VS Code. Once it's running, open the command palette with Ctrl/Cmd + Shift + P and search for "Claude Code Open in New Tab." You can also click the Claude logo if it's visible in your sidebar.

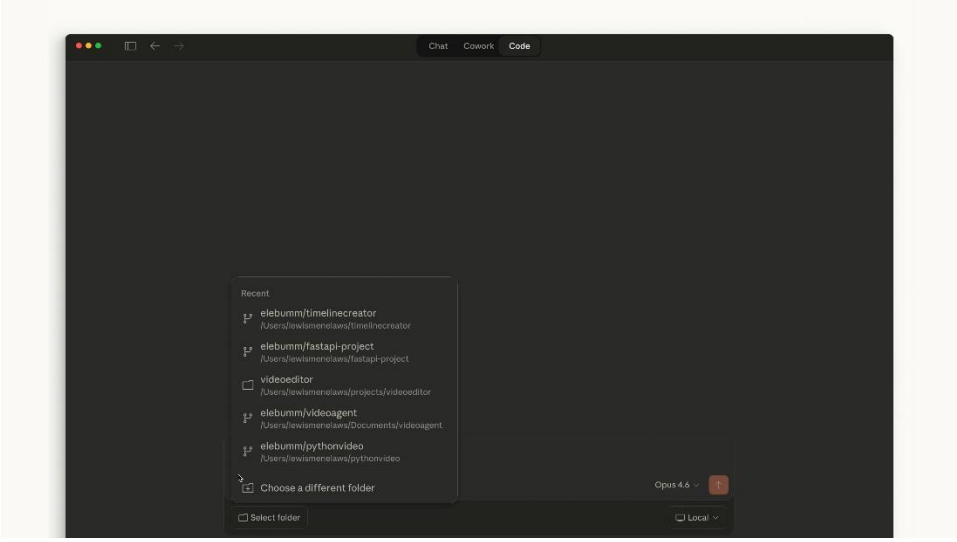


The VS Code extension provides a very similar experience to the terminal. You can also opt out of the UI and use the terminal experience directly in your settings.

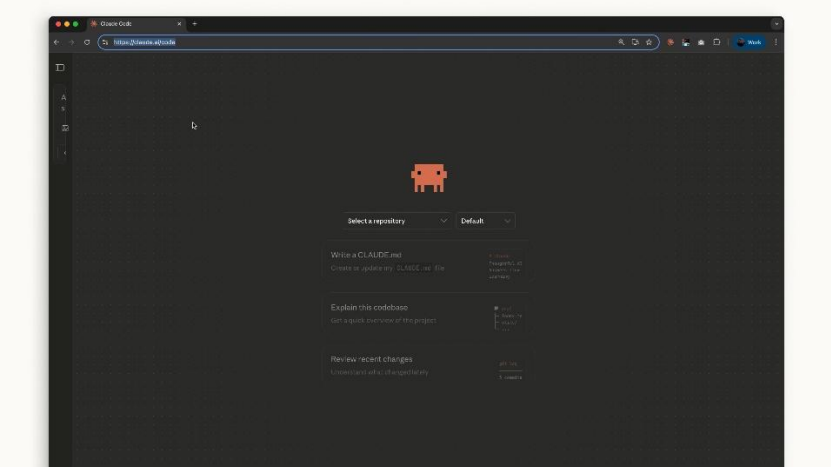
JetBrains: Install the Claude Code plugin from the JetBrains Marketplace. After installation, restart your IDE. When you reopen it, you'll see the Claude logo. Clicking it opens a pane with the terminal experience that works alongside your editor.



Desktop: After installing and signing into Claude Desktop, you'll see a toggle at the top labeled "Code." The look and feel is similar to the chat side of things, but it lets you work in a specific folder, change permissions, and even work in a cloud environment.



Web: On the web, access Claude Code by going to claude.ai/code, or by clicking the "Code" label in the sidebar of the chat app. This works similarly to the desktop app, but you're restricted to GitHub repositories.



Which One Should I Use?

If you want to stay on the cutting edge, the terminal is your best bet — features ship there first. The IDE integrations offer a nearly identical experience if you prefer Claude Code to feel more intertwined with your code editor.

Desktop is great for letting Claude run in the background while you handle other tasks.

Claude Code on the web is a solid option if you want to remotely work on projects through a GitHub repository.

However you want to use Claude Code is up to you.

Video

Your First Prompt

[Your first Claude Code prompt]

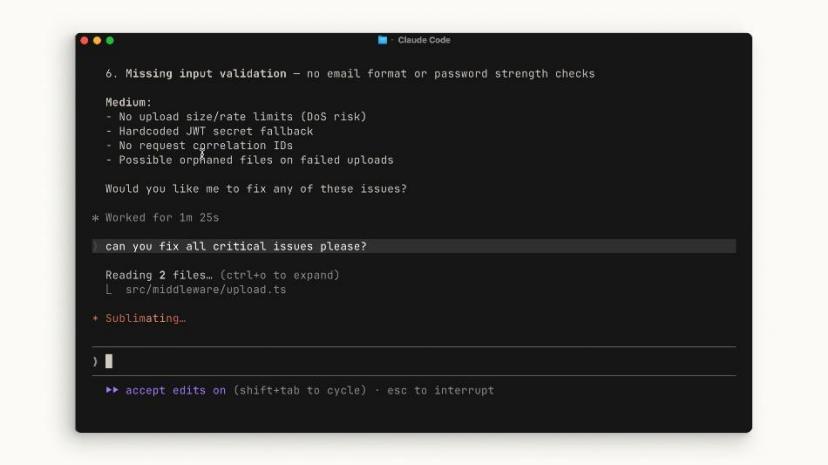
You talk to Claude Code like you would any AI assistant. When entering your prompt, here are some things to consider that can both protect you and make things easier.

Auto-Accept vs. Approval

You can choose whether Claude auto-accepts every file change it suggests, or whether it asks for your explicit permission each time. Press Shift + Tab to cycle between modes.

- **Approval mode:** Claude asks permission each time it wants to edit a file or run a command.
- **Auto-accept mode:** File edits are automatically approved, but commands still require your permission.

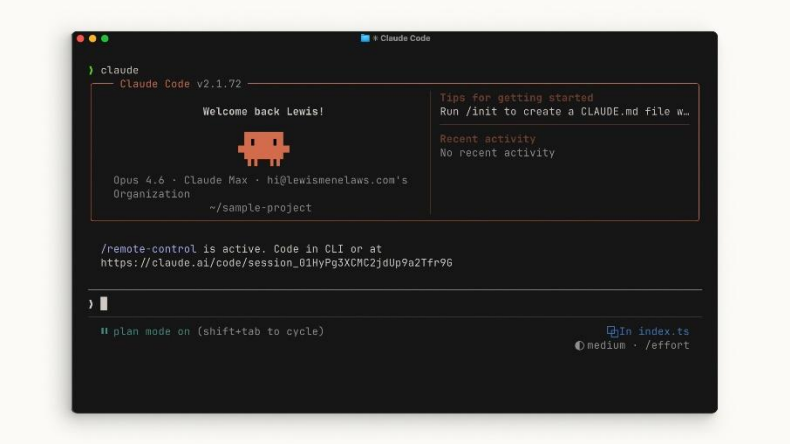
There's no right or wrong answer — it's whatever you're comfortable with.



Plan Mode

Within the Shift + Tab menu is **Plan Mode**. **Plan mode** takes your prompt and uses read-only tools to analyze your codebase and research your suggested implementation. It will ask clarifying questions along the way, then return a detailed plan it can execute.

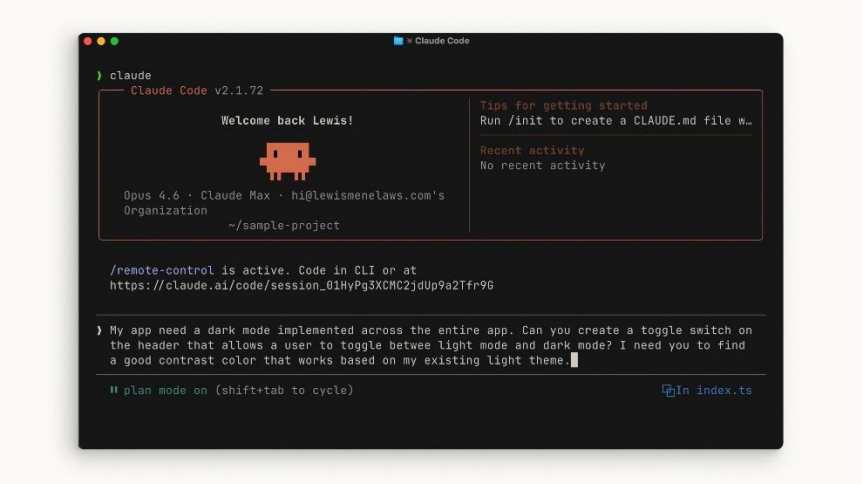
Plan mode is great for planning complex changes or doing a safe code review. Many times you'll be asking Claude to handle multi-step implementations toward a feature, and this is exactly where **Plan Mode** excels.



Example: Add a Dark Mode Toggle

Let's walk through an example. Say you have an application that needs a dark mode toggle. Open the root directory of your project and run Claude. Press **Shift + Tab** a couple of times to enter **Plan Mode**, then write a prompt like:

My app needs a dark mode implemented across the entire app. Can you create a toggle switch on the header that allows a user to toggle between light mode and dark mode? I need you to find a good contrast color that works based on my existing light theme.



Let Claude plan it out. After reviewing the plan, if it looks good, accept it and let Claude ask you for approval at each step. At the end, you can see exactly what Claude did and how it reached its conclusions.

Recap

When using Claude Code, try to be as descriptive as possible with your prompt. If you want to stay in the loop at every step, you can. Use Plan Mode to let Claude dig into the details of what you want to achieve before executing on any code.

The Explore → Plan → Code → Commit Workflow

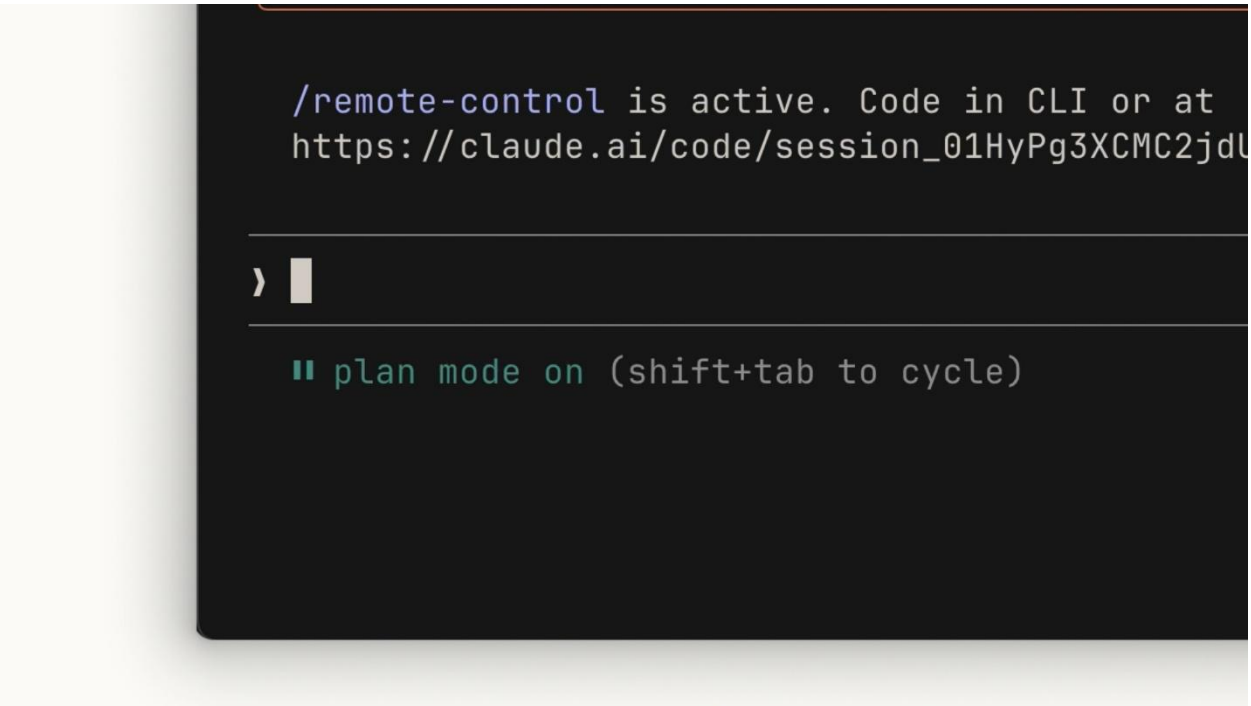
[The Explore → Plan → Code → Commit workflow in Claude Code]

If you take one thing away from this course, let it be this workflow: Explore, Plan, Code, and Commit. Without it, most people jump straight to asking Claude to write code — which means more course-correcting later on.

Explore and Plan

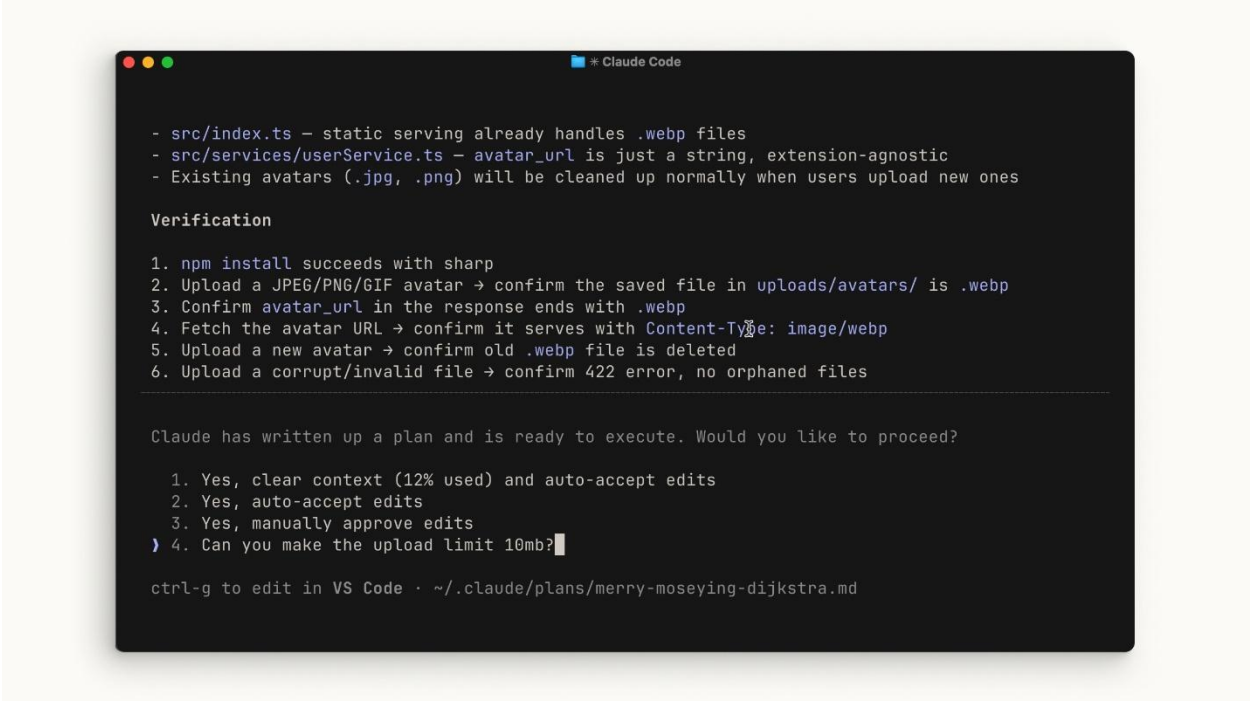
The fastest way to handle these first two steps is with Plan Mode. In plan mode, Claude can't edit files — it just reads files to gather information about how it will tackle the implementation.

To enter plan mode, press Shift + Tab until you see "Plan Mode" under the text input. Then write a prompt like:



I need to add WebP conversion to our image upload pipeline. Figure out where in the pipeline it should happen, whether we need new dependencies, and how to approach it.

Claude will read relevant files, run some web searches, and give you a plan of action. Review it and decide if it meets your criteria. If not, ask it to revise specific areas.



This is the best place to course-correct because it's before any code is written. You can also run the explore subagent without being in plan mode if you just want a general summary of your codebase without intending to make changes afterward.

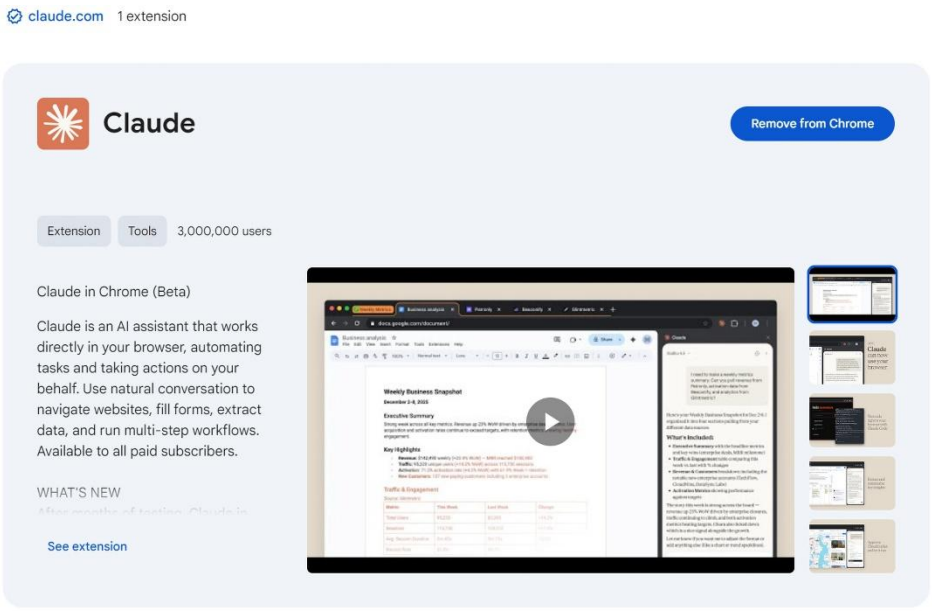
Code

Once the plan looks good, select "approve" to accept it and let Claude work through the list items. You can choose whether Claude auto-accepts file edits or asks you each time.

Claude will do its best to troubleshoot before considering the plan "finished," but at times you'll need to step in. This is the benefit of working with Plan Mode — after execution, you also have the context of how you got to the results, which helps guide Claude's next decisions.

A few tips to make the coding phase smoother:

- Define a success criteria. For Claude to be confident in its results, it needs to be clear on what "correct" looks like. Make this explicit when writing your plan.
- Add tools. Tools that help Claude complete its goals remove a lot of back and forth. For example, if you're building web UIs, install the Claude in Chrome extension so Claude Code can control a browser tab and test the UI directly.



Details

- Include a test suite. Give Claude a test suite it can continuously validate against. Claude can even write tests for you. Before handing this off, make sure the tests are a reliable source of truth to avoid false positives.

Quick tip: If you find Claude keeps running into the same issues, ask it to save the solution to its CLAUDE.md file.

Commit

Once you've tested the changes yourself and are happy with the results, it's time to push your code. Before you commit, run a subagent code reviewer to look at your work. A subagent gets a fresh pair of eyes on the codebase — it doesn't carry the bias the main agent might have from the session.

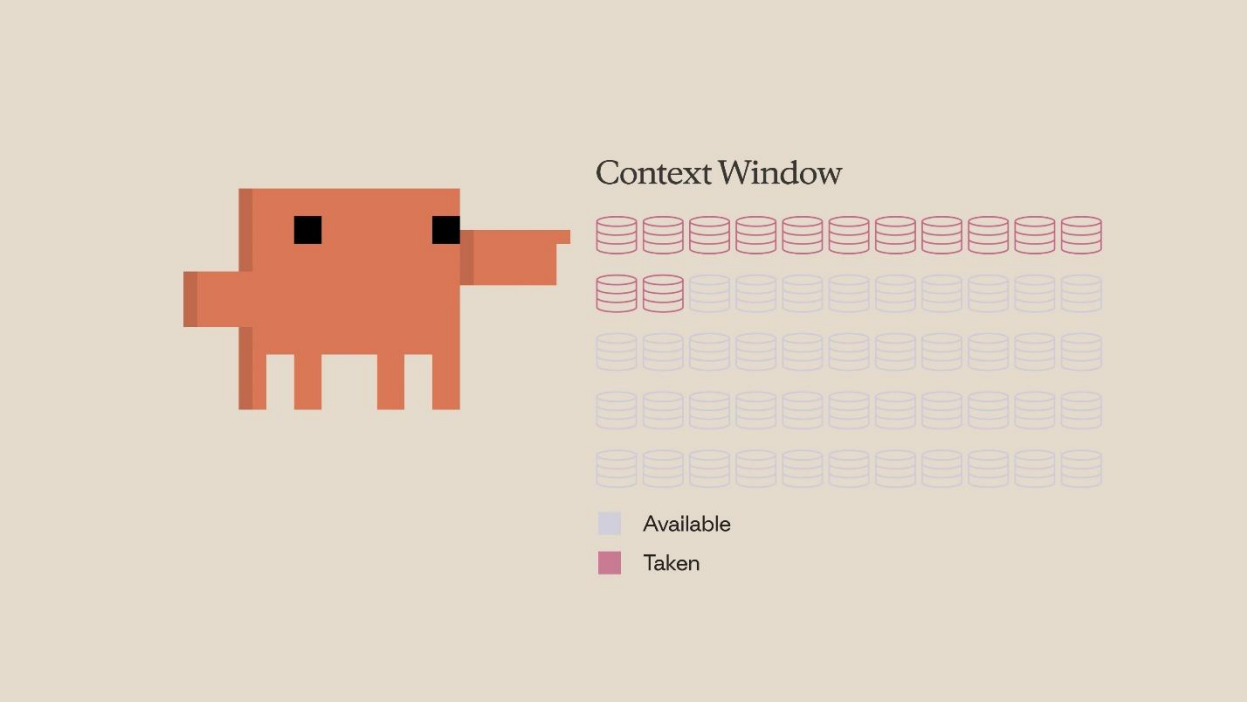
Context Management

Context Management in Claude Code

Context is Claude's working memory. Every file it reads, every command it runs, every message you send — it all takes up space in the context window.

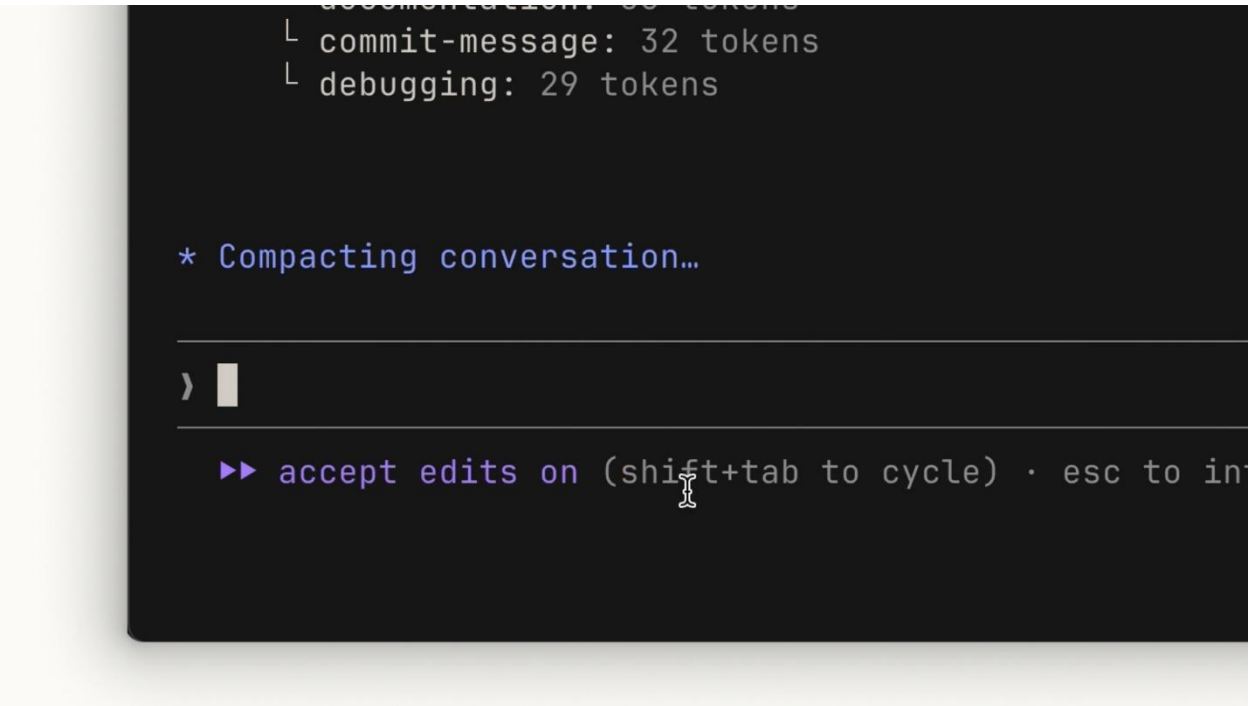
What is the Context Window?

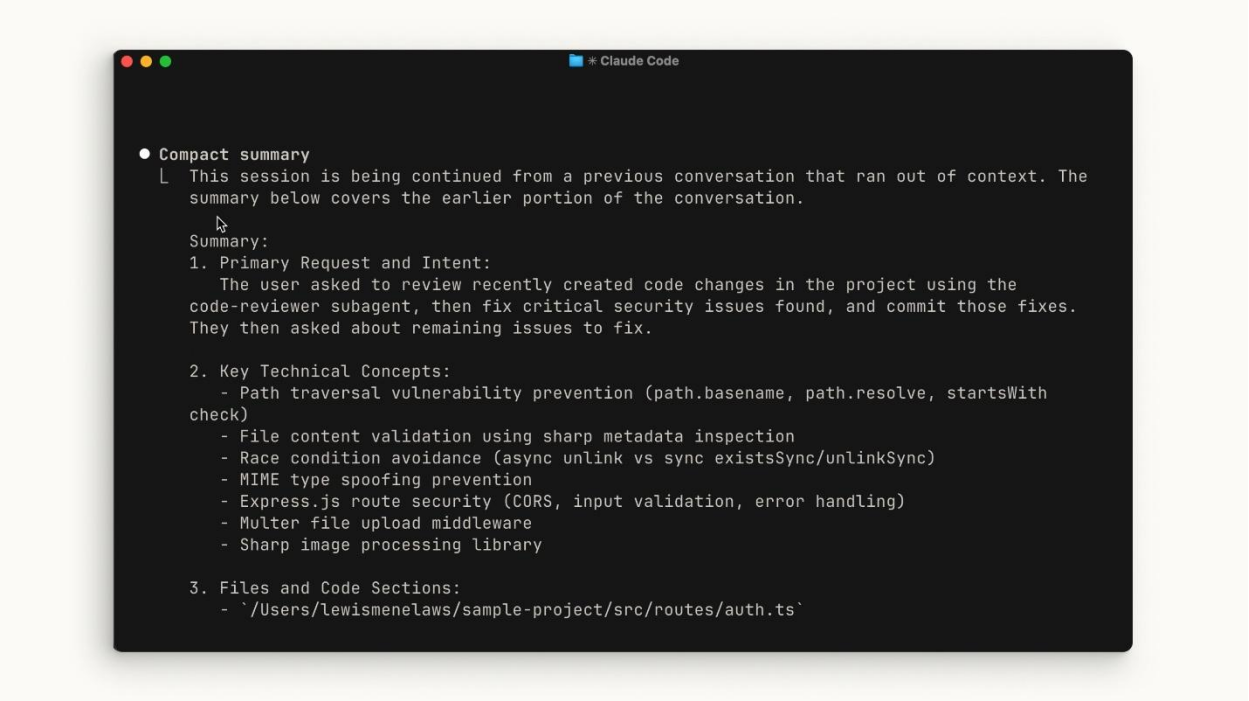
Think of the context window as the amount of space Claude can hold in its memory. Whenever you enter a prompt, Claude reads a file, runs a tool call, or receives a tool call result, it's all adding to the context window. Since there's a finite amount of space, it becomes important to optimize how you use it.



What Happens When Context Fills Up

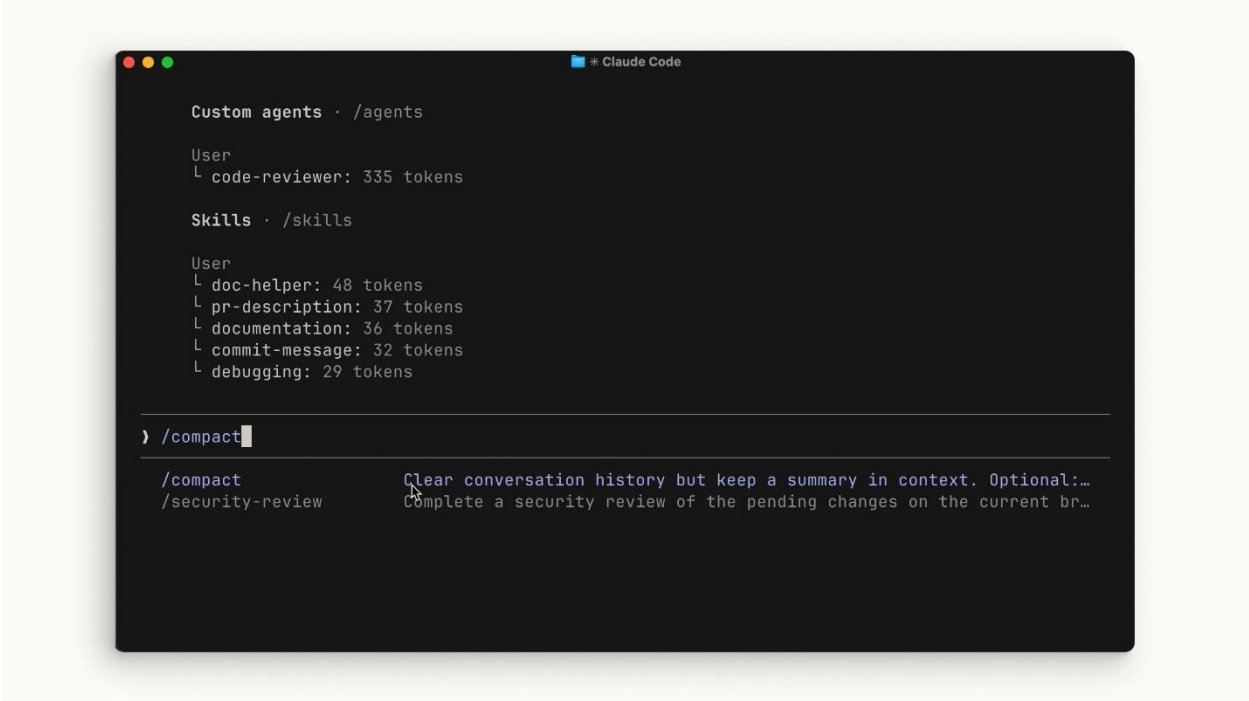
When you approach the limit, the context window is automatically compacted. Compaction summarizes important details and removes unnecessary tool call results to free up space. Note that this process can potentially lose details.



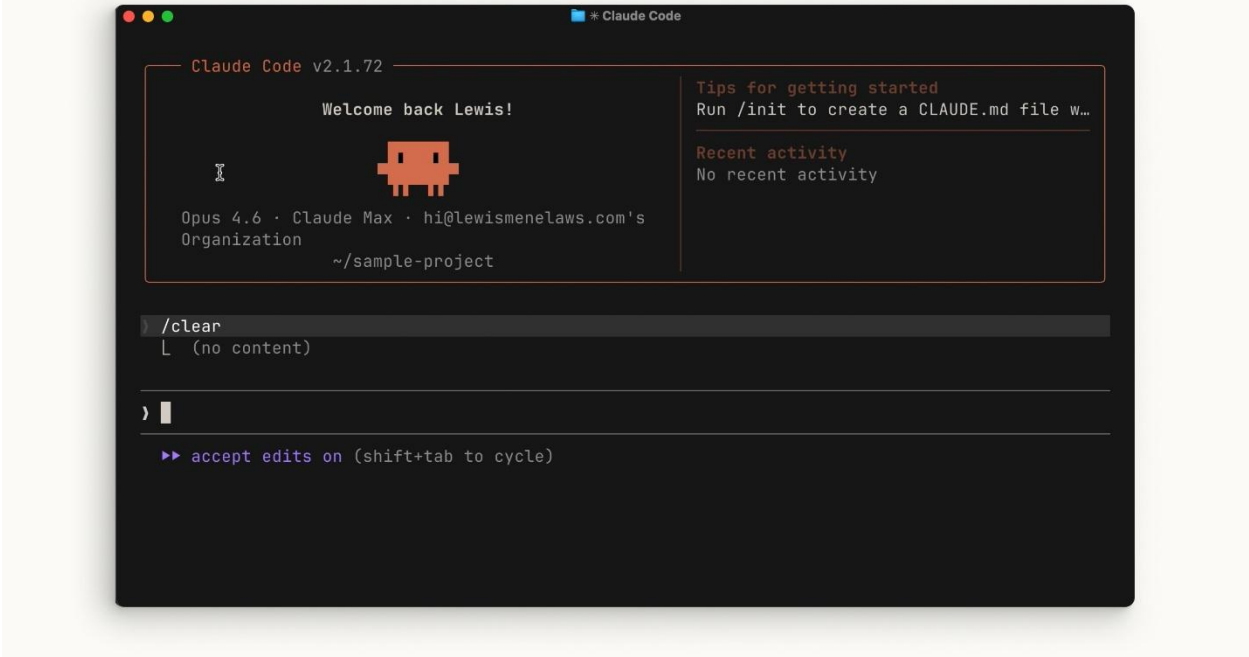


Commands

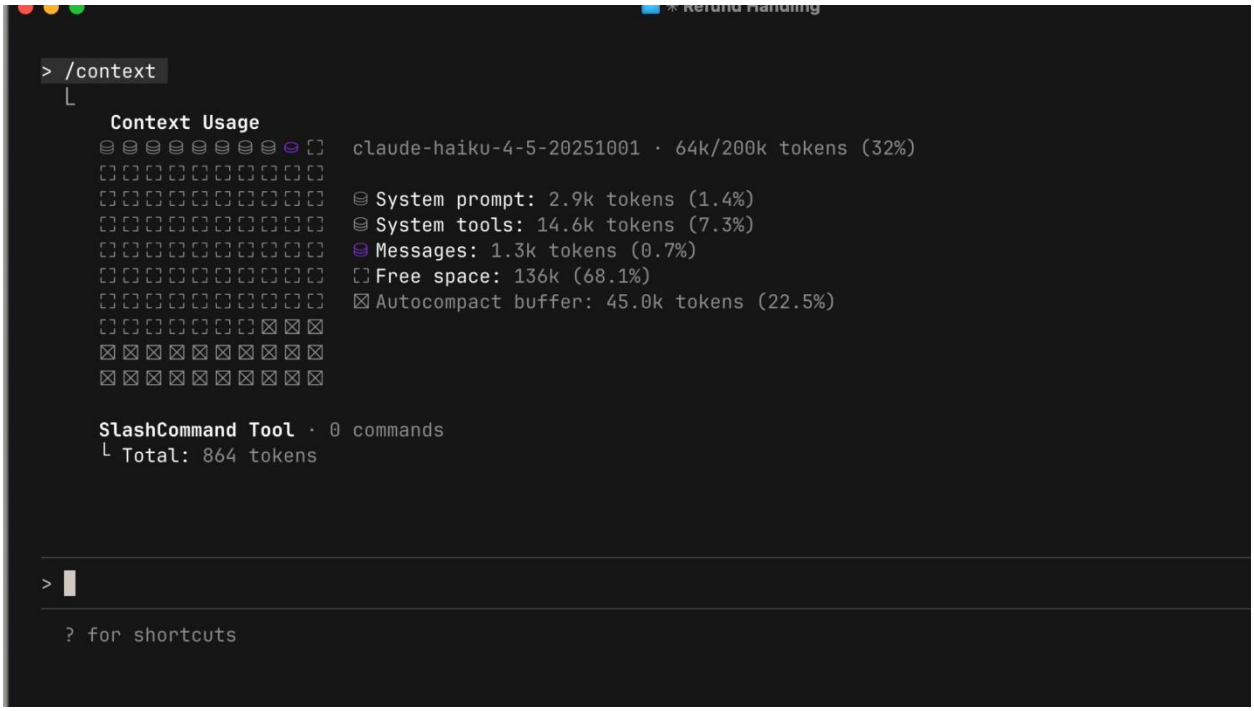
You can run compaction manually with the /compact command. This compacts everything up to that point. It's handy when you want to free up context space while keeping a memory of what you previously worked on.



If you want to completely start from scratch with no memory of the previous session, run /clear. This removes everything.



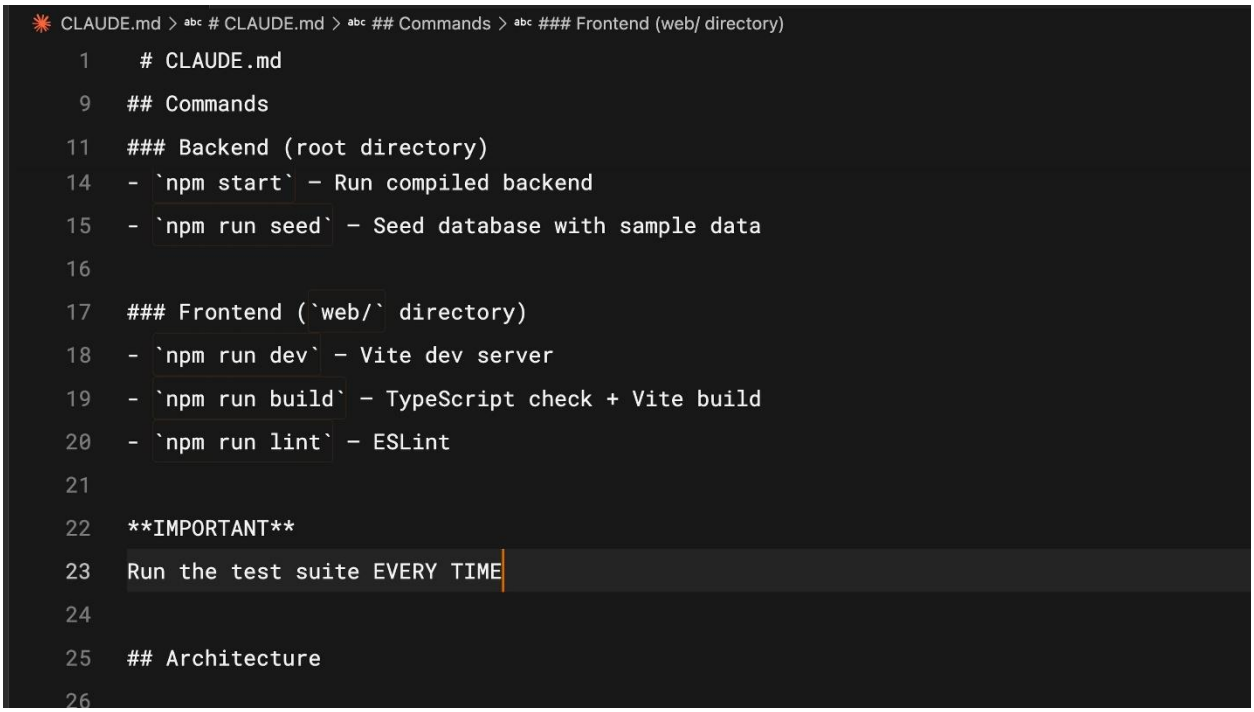
To check the state of your context, run the /context command. You'll get a high-level overview of your context size, the categories taking up the most space, and a visual graphic showing the breakdown.



When to Use Which

A general rule of thumb:

- Use /compact when you're working on a specific feature and running up against the context limit but need to continue. Keeping the context relevant to your current feature is important.
- Use /clear when you want to start a new feature. You don't want the previous conversation to introduce bias into something new. For things you want Claude to remember across sessions, put them in your CLAUDE.md file so it doesn't have to rediscover things from scratch.



Tips for Saving Context Space

Be specific. A vague prompt might seem smaller, but it actually costs more context in the long run. Without clear instructions, Claude is forced to explore your codebase more and do its own reasoning — which takes up far more context space than a detailed prompt would.

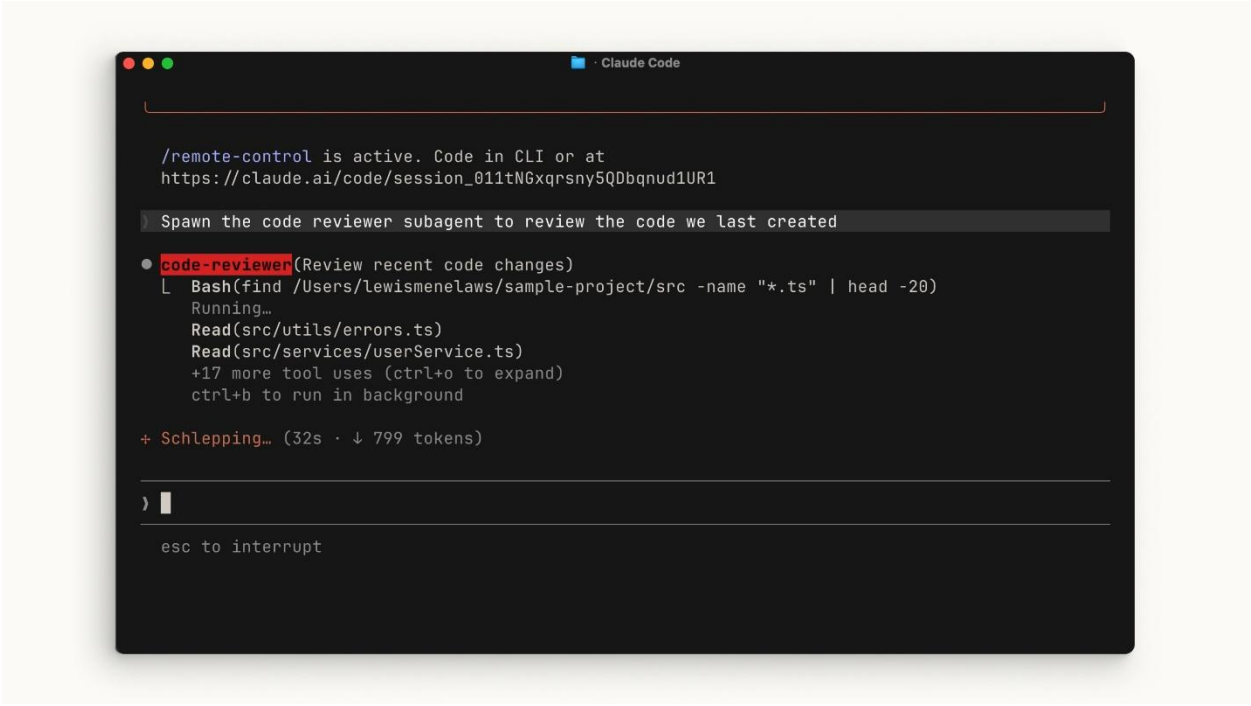
Manage your MCP servers. MCP servers load all of their available tools into context by default, even when you're not using them. If you have servers configured for things unrelated to the current project, consider turning them off. You can also try "Skills," which work similarly to MCP servers but don't load everything into context upfront.

Use subagents. Subagents run in parallel with your main agent but have a completely separate context window. For tasks where you only need the answer — like "where are the authentication endpoints located?" — a subagent does the work and returns just a summary to your main agent, keeping your primary context clean.

Recap

Managing context within Claude Code is crucial. Use /compact to summarize long sessions and /clear to start fresh. To use your context window effectively: be specific with

your prompts, check what's consuming your current context, and use subagents to delegate tasks where you only need the result.



Then get Claude to generate a commit message in your style. Rinse and repeat.

Recap

To be effective with Claude Code, follow the Explore, Plan, Code, and Commit workflow:

- Explore gives Claude the relevant context it needs for your project.
- Plan creates a plan of action that Claude uses to measure success.
- Code is the back and forth between you and Claude before settling on the final outcome.
- Commit helps you review and push your code so you can start on your next feature.

Code Review

Video Explainer Milka Jeff Kreo Batch6 9x16 Batch6

Claude Code has a few built-in features that make your git workflow faster. Let's go through them.

Review with a Subagent

Before you push a PR, ask Claude to use a subagent to review your changes. The subagent runs in its own context window with fresh eyes — it doesn't carry the bias of the main agent that just spent the session writing the code.

When creating a code-reviewer subagent, restrict it to read-only tools. A reviewer should flag issues, not edit files. Check the subagent configuration into your repo so your whole team uses the same reviewer.

The /commit-push-pr Skill

The /commit-push-pr skill handles the commit, push, and PR creation all in one step. Instead of doing each manually, just run the skill and Claude takes care of it.

If you have a Slack MCP server configured with channels listed in your CLAUDE.md, it will automatically post the PR link to your team's channel.

Session Linking with --from-pr

When Claude creates a PR through gh pr create, the session gets linked to that PR automatically. If you need to come back to it later — maybe to address review comments or fix a failing build — run:

claude --from-pr <PR_NUMBER>

This picks up right where you left off.

Recap

Use a subagent for an unbiased code review before pushing. Use /commit-push-pr to handle the full commit-to-PR flow in one step. And use --from-pr to resume work on a PR later. These are small features, but they remove a lot of friction from your daily workflow.

Customizing Claude Code

The CLAUDE.md file

The CLAUDE.md File

One of the most useful features in Claude Code is the CLAUDE.md file. It gives Claude Code persistent memory about your project.

The Problem It Solves

When you open Claude Code without a CLAUDE.md file, it starts fresh every time. It has to re-explore your codebase, figure out what dependencies are needed, and understand what features are already implemented. Sometimes it makes assumptions, which makes it harder to steer Claude in the right direction.

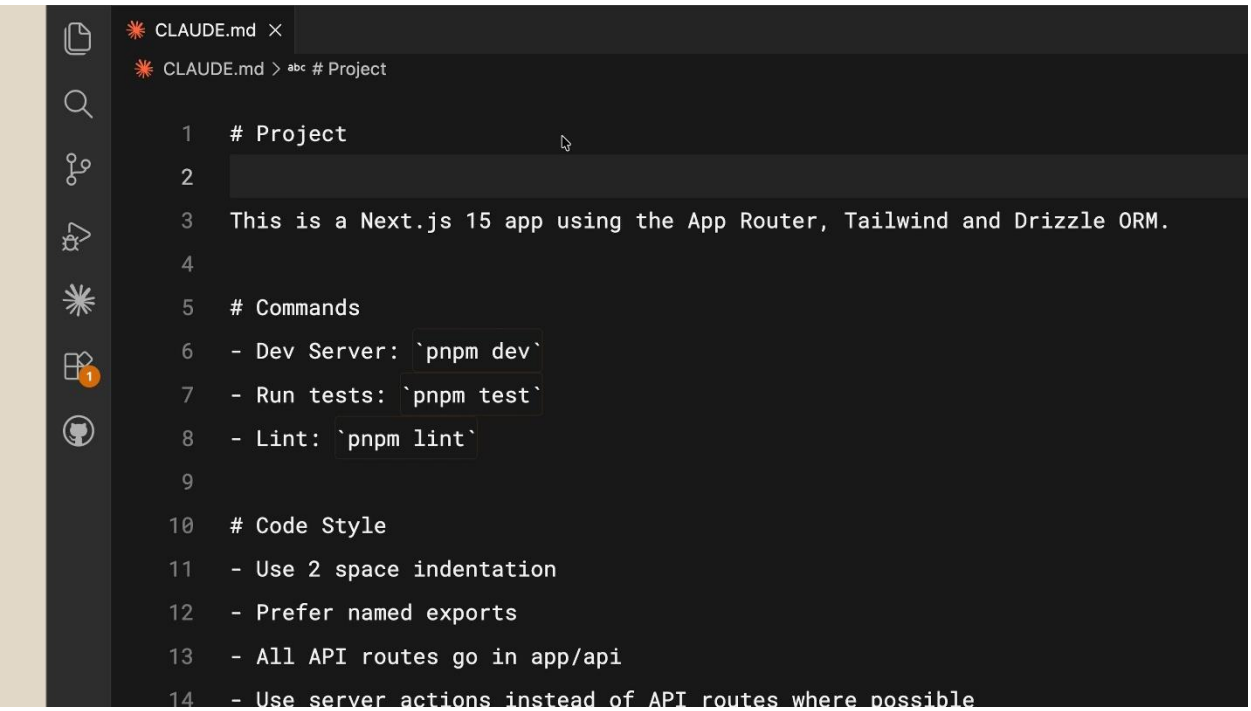
CLAUDE.md solves this. It's a Markdown file you add to the root of your project, and Claude Code reads it automatically every time you start a session. Think of it as an onboarding script for your codebase. The contents of the CLAUDE.md file are appended to your prompt.

An Example

Here's what a typical CLAUDE.md file looks like:

```
# Project

This is a Next.js 15 app using the App Router, Tailwind, and Drizzle ORM.

# Commands

- Dev server: `pnpm dev`
- Run tests: `pnpm test`
- Lint: `pnpm lint`

# Code Style

- Use 2-space indentation
- Prefer named exports
- All API routes go in app/api/
- Use server actions instead of API routes where possible
```

It's straightforward. Now if you ask Claude Code to create a React component, it already knows to use Tailwind for styling and to follow your code conventions.



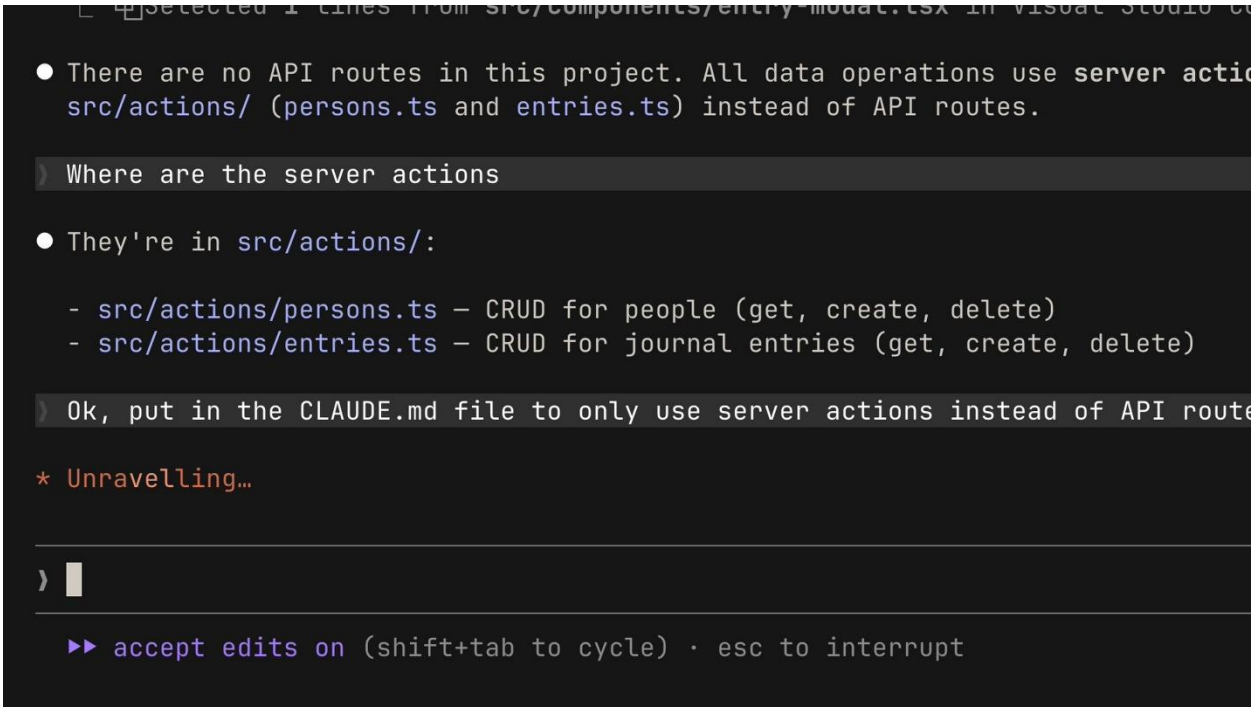
CLAUDE.md is for Teams

You can (and should) commit your CLAUDE.md to version control so your team benefits from it. There's actually a hierarchy of memory files depending on who they're for:

- Project-level CLAUDE.md lives in the root directory of your project. Shared with the team.
- User-level CLAUDE.md lives in your configuration folder. This one is just for you and applies across all your projects. Put your personal preferences here.

Tips

Save corrections to memory. If you find yourself correcting Claude repeatedly — like telling it to always use server actions instead of API routes — explicitly ask Claude to save that rule to memory. Next time you open the project, it'll know.



Reference project docs. If you have documentation in your project that you want Claude to reference, use the @ symbol with the file path:

README.md

Please read if you need more info: @README.md

Start without one. We recommend starting a project without a CLAUDE.md file so you can see where you constantly have to course-correct the model. This keeps your CLAUDE.md compact and focused on only the necessary information. When you're ready, run /init to have Claude generate one for you.

Recap

The difference between a frustrating Claude Code session and a productive one often comes down to context — and the CLAUDE.md file is how you provide that context. Start with your stack, your preferences, and your commands, then build from there as you go.

Subagents

What are subagents?

Claude can delegate tasks to subagents that break them down and run component tasks in parallel, improving your context management. Each subagent operates in its own isolated context window.

How It Works

Managing context in Claude Code is important. A lot of the context window gets consumed by things like tool calls exploring your codebase or running web searches for research. What Claude discovers during that exploration isn't always relevant to the main feature you're developing.

This is where subagents come in. Claude spawns a subagent to handle a task like "explore this codebase for me." The subagent runs in parallel with its own context window, does all the exploration work, and once finished, summarizes its findings and returns that summary back to Claude.

The result: you get the answer you were looking for, without the entire journey it took to get there cluttering your main context.

Creating Your Own Subagent

Subagents are defined in Markdown files with YAML frontmatter. The easiest way to get started is to let Claude generate one for you. Run:

/agents

Then select "Create new agent." You'll walk through steps including choosing the scope of the agent, defining its purpose, selecting the tools it has access to, and even picking a color for it.

Claude will generate a name, description, and prompt for the subagent. This also tells Claude when to call the subagent based on the prompts you give it.

Further Customization

Subagents can be customized further. Here are some highlights:

- Persistent memory lets your subagent retain memory across conversations. This is great if you're using it consistently on the same projects.

- Preload skills into subagents by adding the skill key and listing skills by name. Note that unlike skills in your main conversation, the entire skill is loaded into context here.

Recap

Keeping your context window clean is one of the best ways to stay productive with Claude Code. With subagents, you can run an agent in the background to handle the heavy lifting and return just the answer to your main context window.

Want to go deeper? Check out our dedicated course: Introduction to subagents

Skills:

Presentations | H (14Apr LK)

Article

Want to go deeper? Check out our dedicated course: Introduction to agent skills

MCP

MCP in Claude Code

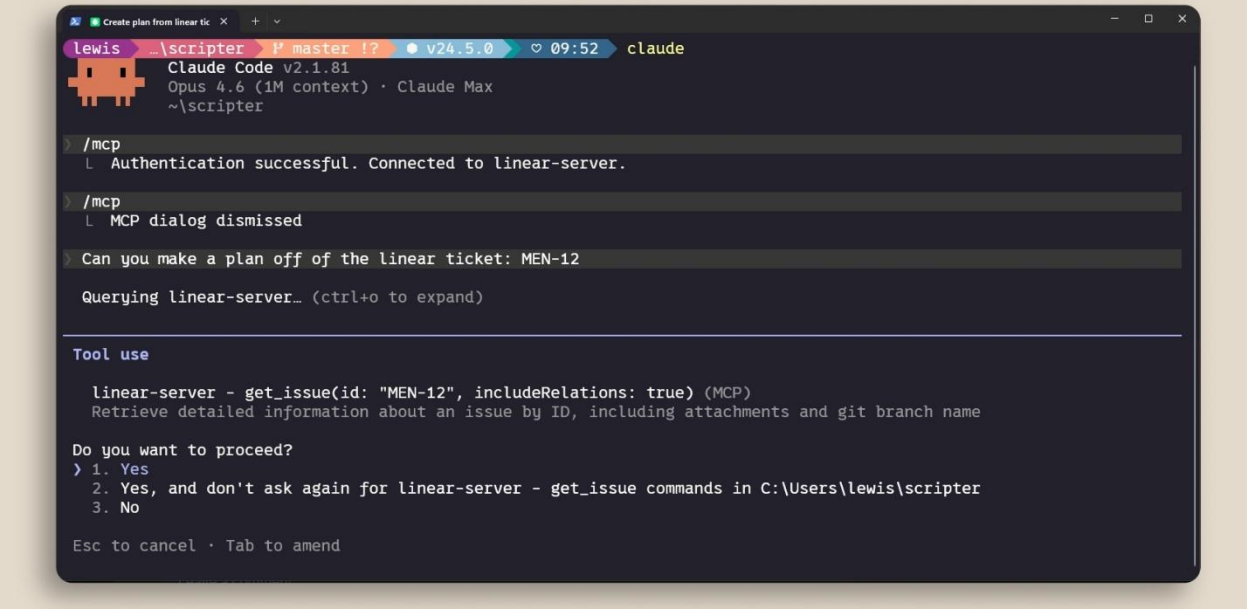
Model Context Protocol (MCP) is an open standard that lets Claude Code connect to external tools and data sources. When you ask a question, Claude automatically understands when it should use those tools to better handle your query.

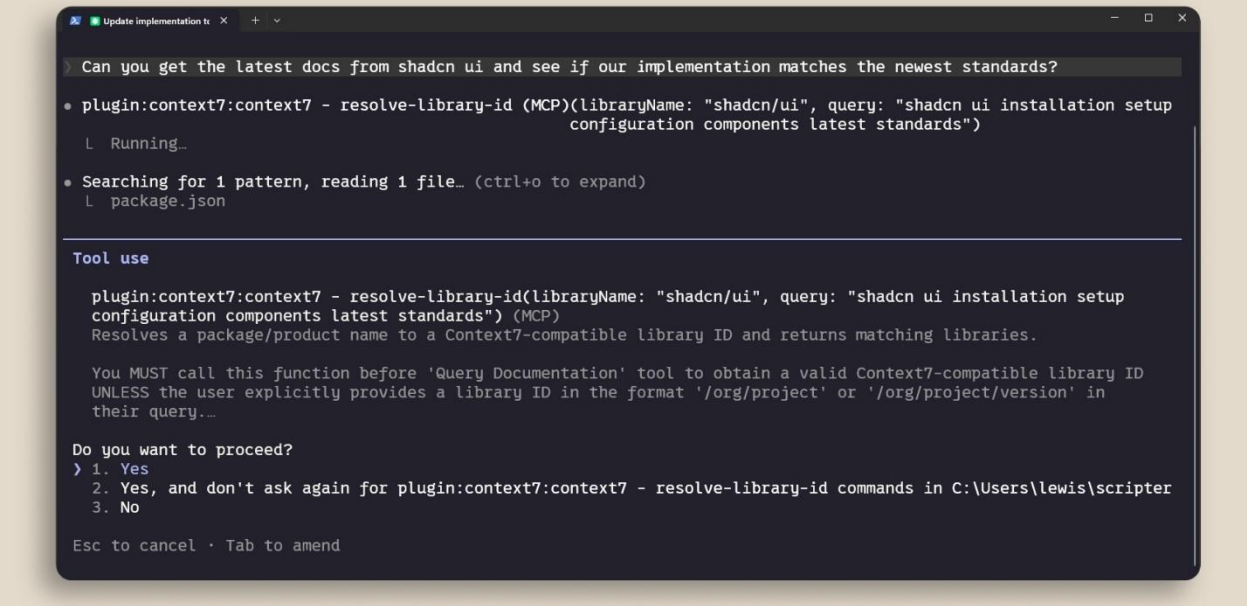
A lot of your context lives outside your codebase — in databases, productivity apps, or public repositories. MCP bridges that gap.

What Can You Do With It?

First, it's important to understand the concept of "tools" in agentic AI. Tools give agents like Claude Code the ability to perform actions that help them complete tasks more effectively. This is different from typical AI, where you just get a text response back.

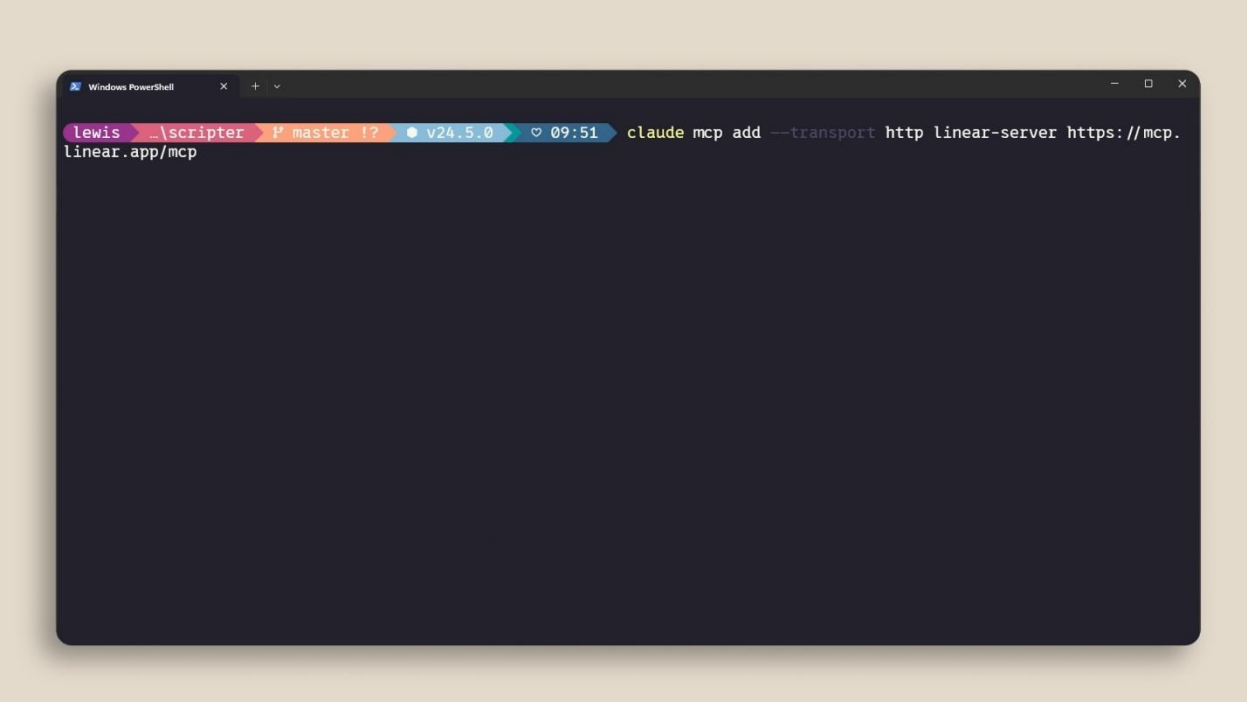
For example, if your team uses Linear for project management, you can add a Linear MCP server to bring in the details of your specific issues. If you need up-to-date documentation for a dependency, a docs MCP server like Context7 can provide that to Claude Code.



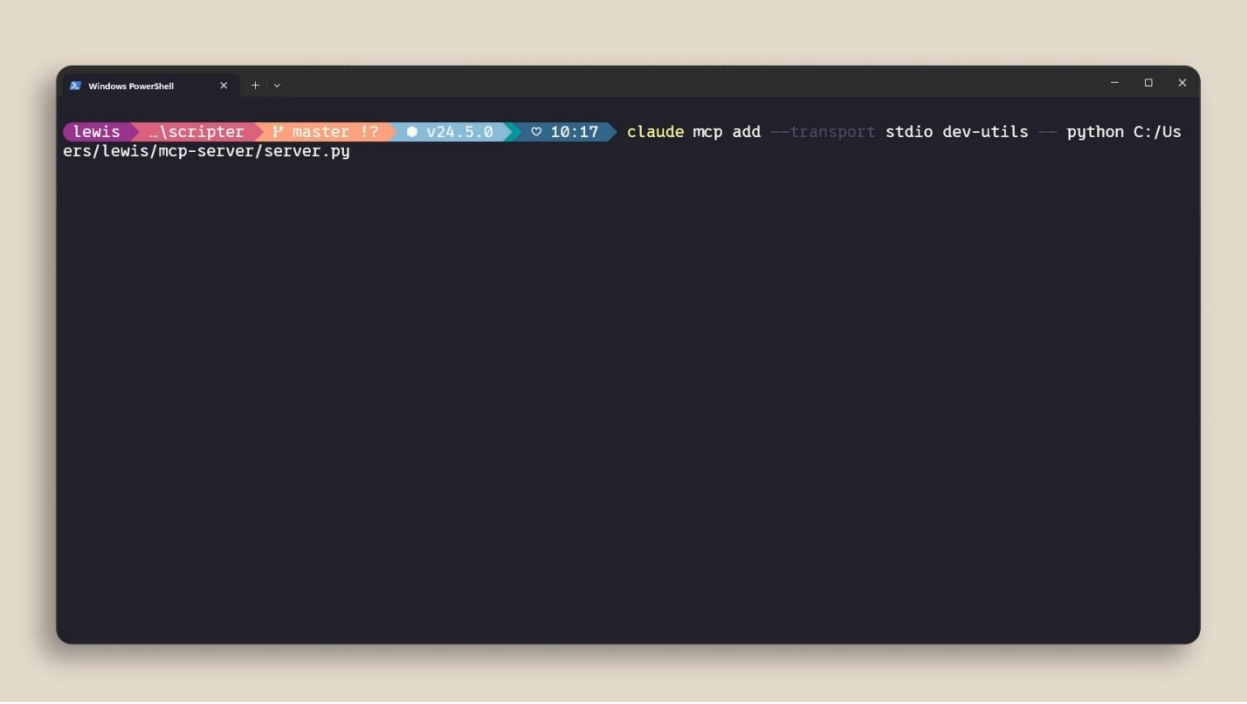


Adding an MCP Server

You can add MCP servers with the claude mcp add command. There are two main types:



- HTTP servers are for remote services. These are hosted by the service provider and connect over the network.
- Stdio servers are for local processes that run on your machine.



You can manage your servers with /mcp inside a Claude Code session to see what's connected, check status, and disable servers you don't need.

Scoping Servers

MCP servers can be scoped in three ways:

1. Local — only available in the current project, just for you.
2. User — available across all your projects.
3. Project — uses a .mcp.json file that you check into version control so anyone on the codebase gets the exact same servers automatically.

Context Costs

MCP servers add tool definitions to your context window — even when you're not actively using them. If you have a lot of servers configured, this eats into your available context. Run /mcp to see what's connected and disable anything you're not actively using.

Hooks

Hooks in Claude Code

Hooks let you run commands at specific points in Claude Code's lifecycle. The key difference between hooks and everything else covered in this course is that hooks are deterministic — they always run.

Why Use Hooks

You can tell Claude in your CLAUDE.md to run Prettier after every file edit. Most of the time it will. But sometimes it won't. A hook makes it happen every single time, no exceptions.

Common use cases include:

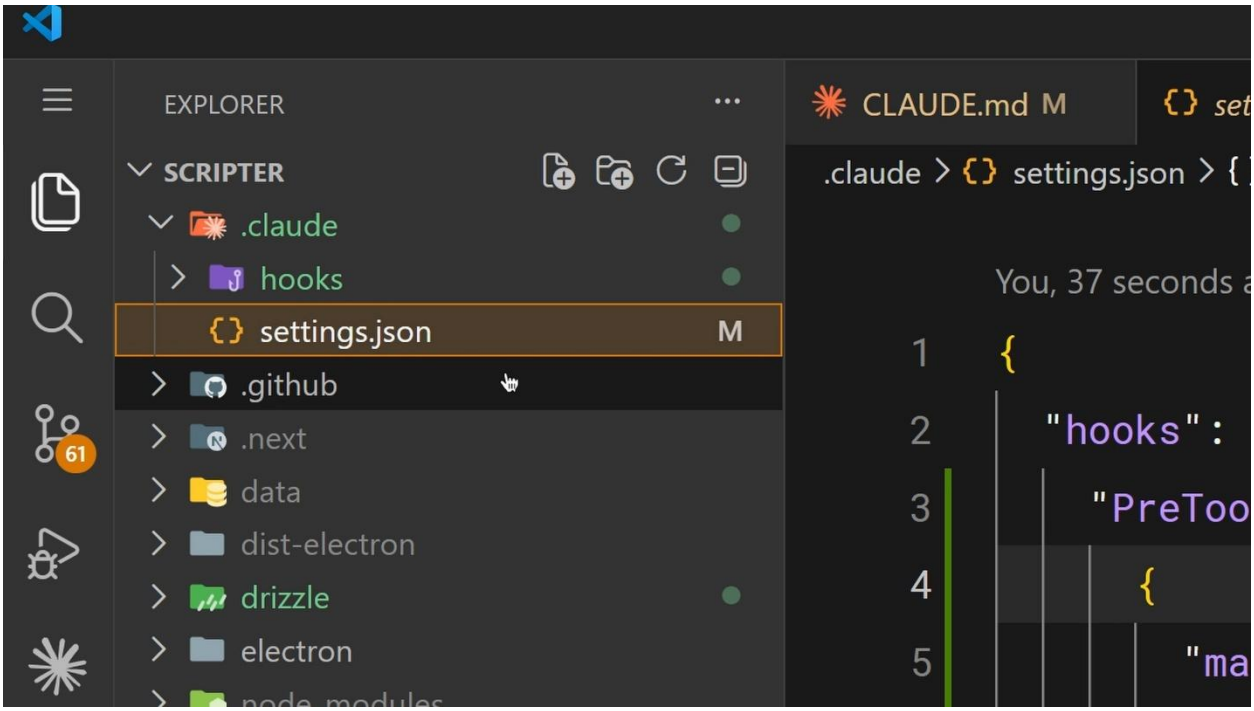
- Auto-formatting after file edits
- Logging all executed commands for compliance
- Blocking dangerous operations like modifying production files
- Sending yourself notifications when Claude finishes a task

How They Work

Hooks are configured in your settings.json. You pick an event, optionally set a matcher for which tools it applies to, and provide a command to run. The available events are:

- PreToolUse — runs before a tool call
- PostToolUse — runs after a tool call completes
- UserPromptSubmit — runs when you submit a prompt, before Claude processes it
- Stop — runs when Claude finishes responding
- Notification — runs when Claude sends a notification

You configure them through the /hooks command inside Claude Code, or by editing settings.json directly.



A Practical Example

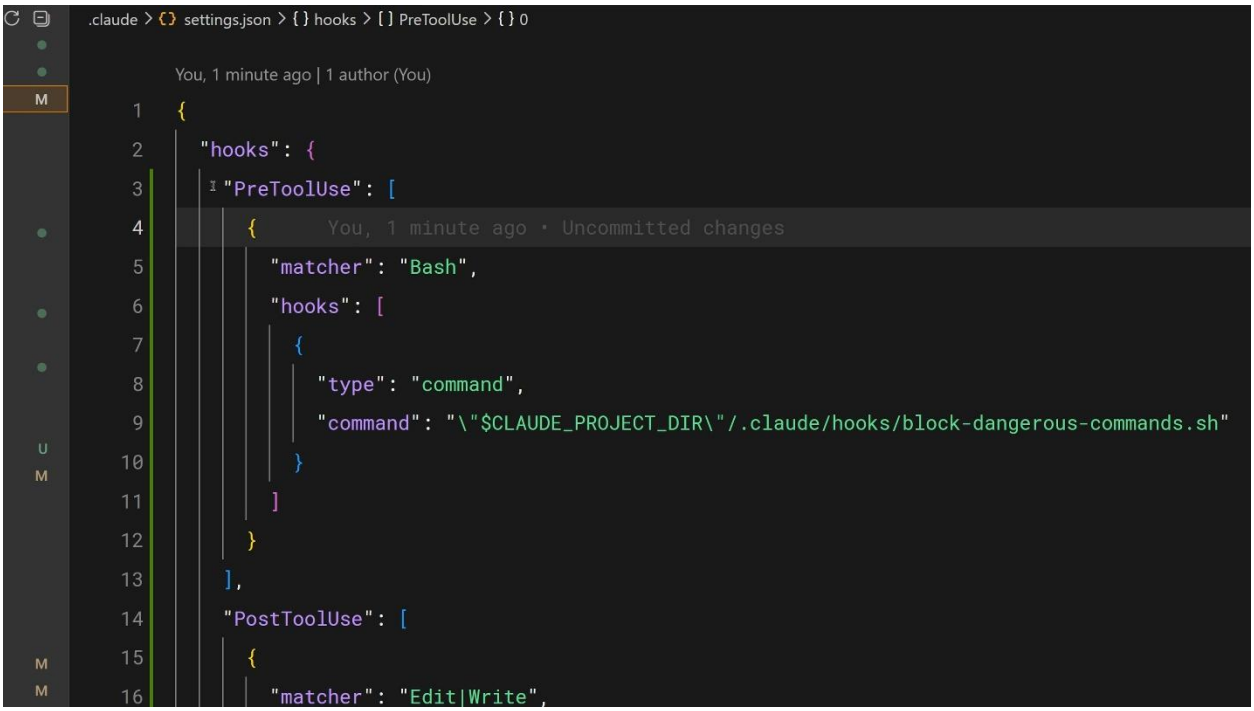
The most common hook: auto-formatting after edits. Set a PostToolUse hook with a matcher of "Edit|MultiEdit|Write" so it fires whenever Claude modifies a file. The command checks the file extension and runs the appropriate formatter — Prettier for TypeScript, gofmt for Go, whatever your project uses.

Blocking with PreToolUse

PreToolUse hooks can block tool calls before they execute. Your hook receives the tool name and input as JSON on stdin. The exit code determines the behavior:

- Exit code 0 — proceed normally.
- Exit code 2 — block the action. The stderr message gets fed back to Claude as feedback so it knows why it was blocked and can adjust.
- Any other exit code — a non-blocking error that gets shown to you but doesn't stop anything.

This is how you enforce hard rules. Block writes to a production config directory. Block bash commands that contain rm -rf. Block commits to main. Whatever your team needs to be *guaranteed*, not suggested.



Sharing Hooks with Your Team

Hooks configured in .claude/settings.json are project-level and can be checked into your repo. This means your entire team gets the same hooks automatically. Use the CLAUDE_PROJECT_DIR environment variable in your commands to reference scripts stored in your project, so they work regardless of Claude's current working directory.

Recap

Hooks give you deterministic control over Claude Code's behavior. Use PostToolUse for auto-formatting and logging. Use PreToolUse to block dangerous operations. Configure them with /hooks or in settings.json. And check them into your repo so your team gets them too.

If something needs to happen every time without fail, don't put it in a prompt. Put it in a hook.

If a tool has a CLI equivalent (like gh for GitHub or aws for AWS), the CLI is more context-efficient because it doesn't add persistent tool definitions.

You might also benefit from using a Skill instead. A Skill has a name and description loaded into context, and Claude only loads the full skill contents when it determines it needs to use it.

If your MCP tools exceed 10% of your context window, Claude Code automatically switches to tool search mode, which discovers the right tools on demand — though this may not work as reliably.

Recap

MCP connects Claude Code to your external tools and data sources. Add servers with claude mcp add. Scope them to your project with .mcp.json so your team gets them automatically. And keep an eye on context usage by disabling servers you're not actively using.